

Supplementary Study Guide

Topics Beyond Python Crash Course
For the Multi-Threaded Data Aggregator Capstone

Written as original teaching material, sequenced to match Python
Crash Course's chapter order. Each section links to the official
documentation for deeper reference.

How to Use This Guide

This is a **reference you grow into**, not something to read cover to cover today. Each section is self-contained and written for someone who just finished the matching part of *Python Crash Course* (PCC). A note at the top of each section tells you when you're ready for it.

Everything here is written from scratch as a teaching explanation — not copied from any book or site — so you can read it freely, and I'd still encourage you to visit the official docs linked at the end of each section once a topic clicks, since they're the authoritative reference.

1. Exceptions — `try` / `except`

Ready when: you've finished PCC Chapter 10 (Files and Exceptions).

The idea

Normally, an error in your code crashes the whole program. `try` / `except` lets you say “attempt this risky thing, and if it fails in a specific way, do something else instead of crashing.”

```
try:
    number = int(input("Enter a number: "))
    result = 10 / number
    print(result)
except ValueError:
    print("That wasn't a valid number.")
except ZeroDivisionError:
    print("Can't divide by zero.")
```

Why *specific* exception types matter

You could write `except Exception:` to catch anything, but that hides bugs you didn't anticipate. Catching `ValueError` vs `ZeroDivisionError` separately means you can respond appropriately to each, and anything truly unexpected still crashes loudly (which is what you want while learning).

Pattern you'll see constantly in networking code

```
import requests

try:
    response = requests.get("https://example.com", timeout=5)
    response.raise_for_status() # raises an error if status is 4xx/5xx
except requests.exceptions.Timeout:
    print("The request took too long.")
except requests.exceptions.ConnectionError:
    print("Could not connect - check your internet or the URL.")
except requests.exceptions.HTTPError as e:
    print(f"Server returned an error: {e}")
```

Each `except` line catches one *category* of problem. `as e` captures the actual error object so you can print details about it.

Official reference: <https://docs.python.org/3/tutorial/errors.html>

2. Classes & OOP Basics

Ready when: you've finished PCC Chapter 9 (Classes).

You already have this from PCC, but here's the one extra concept the capstone assignment needs beyond what the book covers:

`__slots__`

Normally, any Python object can have new attributes added at any time:

```
class Dog:
    def __init__(self, name):
        self.name = name

d = Dog("Rex")
d.age = 3          # totally fine, Python allows this
d.oops = "typo"    # also "fine" - Python won't warn you about a typo like this
```

That flexibility costs a little memory per object (Python keeps a hidden dictionary of attributes for each instance) and can hide typos. `__slots__` fixes both:

```
class Dog:
    __slots__ = ("name", "age")    # ONLY these attributes are allowed

    def __init__(self, name, age):
        self.name = name
        self.age = age

d = Dog("Rex", 3)
d.oops = "typo"    # raises AttributeError immediately - catches the bug
```

Use `__slots__` when you're creating many instances of a simple class and want to save memory or catch attribute typos early. Skip it for ordinary small projects — it's an optimization, not a requirement.

Official reference: <https://docs.python.org/3/reference/datamodel.html#slots>

3. Working with APIs (`requests` + JSON)

Ready when: you've finished PCC Chapter 17 (Working with APIs) — PCC actually covers the basics of this well. This section fills in a few gaps.

The core pattern

```
import requests

response = requests.get("https://api.github.com/users/octocat", timeout=5)
response.raise_for_status()    # crash loudly if the request failed
data = response.json()        # turns the JSON text into a Python dict
print(data["login"])
```

- `response.status_code` — the HTTP status number (200 = OK, 404 = not found, 500 = server error)
- `response.json()` — parses the response body as JSON into a dict/list
- `response.text` — the raw response body as a string (useful for HTML, not JSON)
- `timeout=5` — give up after 5 seconds instead of hanging forever

Passing query parameters properly

Instead of building URLs by hand with string concatenation, pass a dictionary:

```
params = {"q": "python", "sort": "stars"}
response = requests.get("https://api.github.com/search/repositories", params=params)
```

`requests` handles the URL-encoding for you.

Official reference: <https://requests.readthedocs.io/en/latest/user/quickstart/>

4. Web Scraping with BeautifulSoup

Ready when: comfortable with `requests` (section 3) and dictionaries/loops. PCC doesn't cover this at all, so go slowly here.

What scraping actually is

An API gives you clean JSON. A regular website gives you HTML meant for a browser to *render*, not for a program to read directly. BeautifulSoup parses that HTML into a searchable tree so you can pull out just the pieces you want.

Basic usage

```
import requests
from bs4 import BeautifulSoup

response = requests.get("https://example.com", timeout=5)
soup = BeautifulSoup(response.text, "html.parser")

# Find the FIRST matching element
title_tag = soup.find("h1")
print(title_tag.text)

# Find ALL matching elements
all_links = soup.find_all("a")
for link in all_links:
    print(link.get("href"))
```

Searching by CSS class

HTML elements often have a `class="..."` attribute used for styling. You can search by it:

```
# <span class="price">$19.99</span>
price_tag = soup.find(class_="price")
print(price_tag.text)  # "$19.99"
```

Note the trailing underscore in `class_` — plain `class` is a reserved Python keyword, so BeautifulSoup uses `class_` instead.

Handling missing elements

`find()` returns `None` if nothing matches — it does NOT raise an error. Always check for `None` before using the result, or you'll get a confusing `AttributeError` later:

```
price_tag = soup.find(class_="price")
if price_tag is not None:
    print(price_tag.text)
else:
    print("No price found on this page.")
```

Official reference: <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>

5. Decorators (needed for `@lru_cache`)

Ready when: functions and `*args` / `**kwargs` feel comfortable (PCC Chapter 8).

The core idea

A decorator is a function that wraps ANOTHER function to add behavior, without changing that function's actual code. You've likely already used one without realizing: in PCC's Flask-adjacent examples, `@app.route(...)` is a decorator.

A decorator you write yourself (to build intuition)

```
import time

def time_it(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f"{func.__name__} took {end - start:.4f} seconds")
        return result
    return wrapper

@time_it
def slow_add(a, b):
    time.sleep(1)
    return a + b

slow_add(2, 3)  # prints timing info, then returns 5
```

`@time_it` above `def slow_add` is shorthand for `slow_add = time_it(slow_add)`. Every time you call `slow_add(...)`, you're actually calling the `wrapper` function, which does its extra work and then calls the real function inside.

`functools.lru_cache` — a decorator you don't have to write

```
from functools import lru_cache

@lru_cache(maxsize=128)
def fetch_product(product_id):
    print(f"Fetching product {product_id} from the network...")
    # imagine a slow API call here
    return {"id": product_id, "name": "Widget"}

fetch_product(1)  # prints "Fetching..." then returns the dict
fetch_product(1)  # returns the SAME dict instantly - no print this time!
```

`lru_cache` remembers the return value for each unique set of arguments. Call the function again with the same arguments, and it skips the real work entirely.

Official reference: <https://realpython.com/primer-on-python-decorators/>

6. `map`, `filter`, and `functools.reduce`

Ready when: comfortable with `lambda` and list comprehensions.

These three functions transform whole collections without writing an explicit loop.

`map` — transform every item

```
prices = [10, 20, 30]
doubled = list(map(lambda p: p * 2, prices))
# doubled == [20, 40, 60]

# Equivalent list comprehension:
doubled = [p * 2 for p in prices]
```

`filter` — keep only matching items

```
prices = [10, 20, 30, 40]
expensive = list(filter(lambda p: p > 15, prices))
# expensive == [20, 30, 40]

# Equivalent list comprehension:
expensive = [p for p in prices if p > 15]
```

`functools.reduce` — collapse a list into ONE value

```
from functools import reduce

prices = [10, 20, 30]
total = reduce(lambda acc, price: acc + price, prices, 0)
# total == 60
# Step by step: acc starts at 0
#   0 + 10 = 10
#   10 + 20 = 30
#   30 + 30 = 60
```

`reduce` isn't built in like `map` / `filter` — you import it from `functools`. If a plain `for` loop with a running total feels clearer to you, that's completely valid; `reduce` is just another tool for the same job.

Official reference: <https://docs.python.org/3/library/functools.html#functools.reduce>

7. Concurrency, Part 1: Threading

Ready when: comfortable with functions and basic classes. This is a genuinely hard topic — don't worry if it takes a few passes.

Why bother?

If your program downloads 10 web pages one after another, and each takes 1 second, the total time is 10 seconds — even though your computer is just *sitting there waiting* for each response. Threading lets you start all 10 downloads at roughly the same time, so the total time is closer to 1 second.

This only helps for **I/O-bound** work (waiting on network, disk, etc.) — it does NOT speed up heavy math/computation, due to a Python detail called the GIL (Global Interpreter Lock), which is worth knowing exists but not worth worrying about yet.

`ThreadPoolExecutor` — the easy way to use threads

```
from concurrent.futures import ThreadPoolExecutor
import requests

def fetch_page(url):
    response = requests.get(url, timeout=5)
    return len(response.text)

urls = ["https://example.com", "https://python.org", "https://github.com"]

with ThreadPoolExecutor(max_workers=3) as executor:
    results = list(executor.map(fetch_page, urls))

print(results)  # [page length, page length, page length]
```

`executor.map(function, list_of_inputs)` works just like the built-in `map()`, but runs multiple calls to `function` concurrently across a pool of worker threads, and gives you back the results in the same order you put the inputs in.

Official reference: <https://docs.python.org/3/library/concurrent.futures.html>

8. Concurrency, Part 2: `asyncio`

Ready when: threading (section 7) makes sense. This is usually considered the single hardest topic for people learning Python — take your time, and don't feel behind if it takes weeks to click.

How it's different from threading

Threading uses multiple actual threads managed by the operating system. `asyncio` instead runs everything on ONE thread, but the *code itself* voluntarily pauses at `await` points, letting other tasks run while it waits. It avoids some of the overhead of real threads, and scales better to thousands of concurrent operations — but it requires a different library (`aiohttp` instead of `requests`) and new keywords: `async def` and `await`.

The basic shape

```
import asyncio
import aiohttp

async def fetch_page(session, url):
    async with session.get(url, timeout=aiohttp.ClientTimeout(total=5)) as response:
        text = await response.text()
        return len(text)

async def main():
    urls = ["https://example.com", "https://python.org", "https://github.com"]
    async with aiohttp.ClientSession() as session:
        tasks = [fetch_page(session, url) for url in urls]
        results = await asyncio.gather(*tasks)
    print(results)

asyncio.run(main())
```

Breaking this down: - `async def` marks a function as a **coroutine** — it can contain `await` and must be run specially, not called like a normal function. - `await` means “pause here until this finishes, but let other coroutines run in the meantime.” - `asyncio.gather(*tasks)` starts all the coroutines in `tasks` concurrently and waits for all of them to finish. - `asyncio.run(main())` is the entry point that actually starts the whole async event loop — you call this once, from regular (non-async) code.

Official reference: <https://docs.python.org/3/library/asyncio.html>

9. Unit Testing with `pytest` and `unittest.mock`

Ready when: comfortable with functions and classes.

Why test at all?

As programs grow, it becomes easy to fix one thing and accidentally break another without noticing. Tests are small pieces of code that check “does this function still do the right thing?” — you run them anytime you make a change, and they tell you immediately if something broke.

A first `pytest` test

```
# my_math.py
def add(a, b):
    return a + b
```

```
# test_my_math.py
from my_math import add

def test_add_two_positive_numbers():
    assert add(2, 3) == 5

def test_add_negative_numbers():
    assert add(-1, -1) == -2
```

Run with:

```
pytest test_my_math.py -v
```

`pytest` automatically finds any file starting with `test_` and any function inside it starting with `test_`, runs each one, and reports pass/fail. `assert condition` simply fails the test if `condition` is `False`.

Mocking — faking a network call in a test

You don’t want your tests to actually hit the internet — it’s slow, unreliable, and might not even exist while you’re offline. `unittest.mock.patch` temporarily replaces a real function with a fake one for the duration of the test:

```

from unittest.mock import patch, Mock
import api_fetcher

def test_fetch_product_without_hitting_network():
    fake_response = Mock()
    fake_response.json.return_value = {"id": 1, "name": "Widget"}
    fake_response.raise_for_status.return_value = None

    with patch("api_fetcher.requests.get", return_value=fake_response):
        result = api_fetcher.fetch_product(1)
        assert result == {"id": 1, "name": "Widget"}

```

`Mock()` creates a fake object you can configure to return whatever you want. `patch("module.function", ...)` swaps out the real function just inside the `with` block — outside of it, everything goes back to normal.

Official references: - <https://docs.pytest.org/en/stable/getting-started.html> - <https://realpython.com/python-mock-library/>

10. Profiling with `cProfile`

Ready when: you have a working program and want to know why it's slow.

The idea

Instead of guessing which part of your code is slow, `cProfile` actually measures how much time was spent in every function call.

```

import cProfile

def slow_function():
    total = 0
    for i in range(10_000_000):
        total += i
    return total

cProfile.run("slow_function()")

```

This prints a table showing every function called, how many times, and how much time was spent — both in that function alone (`totttime`) and including everything it called (`cumtime`). Look for the biggest numbers; that's your bottleneck.

Official reference: <https://docs.python.org/3/library/profile.html>

11. Suggested Order to Learn All of This

Matched to where you are in *Python Crash Course*:

After PCC Chapter...	Learn this
8 (Functions)	Section 5 here: Decorators
9 (Classes)	Section 2 here: <code>__slots__</code>
10 (Files & Exceptions)	Section 1 here: <code>try / except</code> in depth
17 (APIs)	Section 3 here: <code>requests</code> deep dive, then Section 4: BeautifulSoup
— (not in PCC)	Section 6: <code>map / filter / reduce</code> — anytime after functions click
— (not in PCC)	Section 7: Threading, then Section 8: <code>asyncio</code> — save for last, hardest topic
— (not in PCC)	Section 9: <code>pytest</code> & mocking — once you have a project worth testing
— (not in PCC)	Section 10: <code>cProfile</code> — only needed once something is slow

There's no rush. Everything in this guide assumes you already know dictionaries, lists, loops, and functions — build outward from there one section at a time, and the capstone assignment will make sense by the time you get to it.